INTRODUCTION TO ARTIFICIAL INTELLIGENCE · LECTURE 1 + LAB 1

# Intelligent Agents: Sense, Decide, Act

What an AI agent is, what makes it rational, and how we specify and build one.

Dr.-Ing. Chukwuka Victor Obionwu

TODAY'S JOURNEY

# Two hours, six stops

| Time | Stop | The question we answer |
|---|---|---|
| 0:00 | Warm-up | Is a robot vacuum an agent? |
| 0:10 | What is an agent? | What does it sense, and what does it do? |
| 0:28 | Rationality | What makes an action a good one? |
| 0:43 | PEAS | How do we describe the problem first? |
| 0:56 | Architectures and code | How are four kinds of agent built? (break at 1:10) |
| 1:15 | Lab 1 (after the break) | Can memory beat luck? |
| 1:50 | Exit quiz | What stuck? |

Every stop has a short exercise. Answers appear on the next click.

WHERE WE ARE GOING

# By the end of today you can…

1. Explain what an agent is, using the words sensor, actuator and percept.

2. Say what the agent function does and how it differs from the agent program.

3. Explain why a rational agent is not the same as an all-knowing agent.

4. Write a PEAS description for a new problem.

5. Name the four agent architectures and build two of them in Python.

No prior AI knowledge is assumed. If a word is new, it gets a NEW WORD box.

HOW WE WORK TODAY

# Four signs to watch for

**YOUR TURN**
Stop and try it on paper or in code. A timer is shown.

**NEW WORD**
A technical term, explained in plain words before we use it.

**QUICK CHECK**
A multiple-choice question. Vote with your fingers: 1 to 4.

**STUCK?**
Raise a hand or write the exact message on the board. Mistakes are the point.

Every answer is hidden until you have committed to one. Guessing and being wrong teaches more than being told.

WARM-UP · YOUR TURN

# Is a robot vacuum an agent?

A small robot drives around a room. When it finds a dirty patch, it cleans it. Nobody steers it. Before we use fancy words, let us look at what it really does.

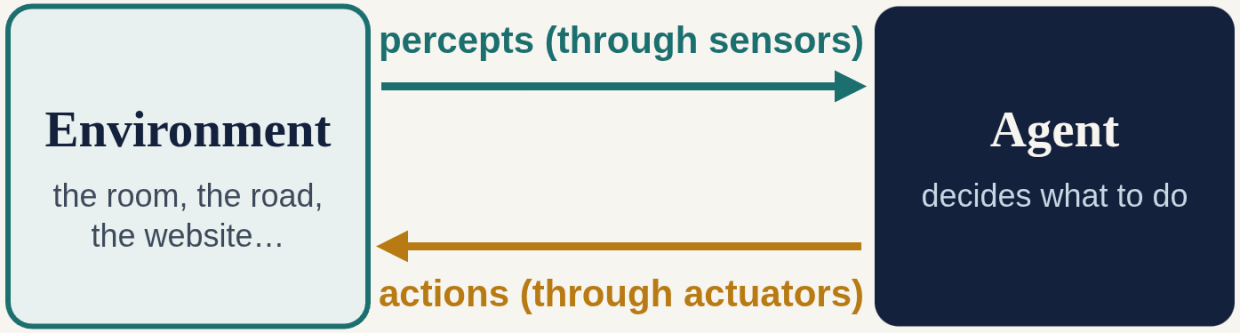
**YOUR TURN (90 seconds)**
On paper, write two lists: (1) every way the robot can **find out** about its room, (2) every thing it can **do** to the room.

**Finding out:** bump sensor, dirt sensor, cliff sensor, wheel counter. **Doing:** drive wheels, spinning brush, suction fan.

The robot takes **information in**, decides, and **acts on the room**, again and again. Anything that does this is called an **agent**. Today is about that loop.

NEW WORD · AGENT

# An agent senses and acts




> **Agent:** anything that perceives its environment through **sensors** and acts on it through **actuators**.

**Sensor:** how it finds out (camera, keyboard, a data feed).

**Actuator:** how it acts (wheel, speaker, a message sent).

**Percept:** what the sensors report at one moment.

AGENTS ARE EVERYWHERE

# Not only robots

| Agent | Sensors (how it finds out) | Actuators (how it acts) |
|---|---|---|
| A person | Eyes, ears, skin | Hands, legs, voice |
| Robot vacuum | Bump, dirt and cliff sensors | Wheels, brush, suction |
| Spam filter | The text of each incoming email | Moving an email to the junk folder |
| Trading program | Live price feed | Buy and sell orders |

Notice the last two have no body at all. A **software agent** senses through data and acts through messages or commands.

QUICK CHECK 1 · VOTE WITH FINGERS

# Which one is an agent?

**A.** A PDF file on a laptop

**B.** A thermostat that reads the room temperature and switches the heater

**C.** A printed timetable

**D.** A calculator with the power off

**B.** The thermostat senses (temperature) and acts (switches the heater) in a loop.

A PDF, a printed page and a switched-off calculator do not sense anything or change anything by themselves.

NEW WORD · PERCEPT SEQUENCE

# What has the agent seen so far?

Our vacuum visits a cell. Each time step its dirt sensor reports one word. Over time it builds a list:




**Percept** = one report right now. **Percept sequence** = the whole list from the start.

**Why keep the history?** At time 3 the percept “clean” alone says nothing about the rest of the room. The sequence tells us we already cleaned the first cell. Memory can change the best action.

THE AGENT FUNCTION

# A rulebook from histories to actions

Mathematicians write the idea in one line. Read it in words first:

$$f : P^* \to A$$

| Symbol | Read it as |
|---|---|
| $P^*$ | every possible percept sequence (the whole history so far) |
| $A$ | the set of actions the agent can take |
| $f$ | the rule that picks one action for each history |

**Example rows of f for the vacuum:** [dirty] → suck · [clean] → move east · [clean, clean] → move east · [dirty, clean, dirty] → suck.

The table never ends, because histories keep growing. So nobody writes it out: we write a short **program** that behaves the same way.

KEY DISTINCTION

# Function versus program

## Agent function

- An abstract idea: what the agent should do for every history.
- A mathematical mapping.
- Says **what**.

## Agent program

- Real code running on a computer or robot.
- Has to be short and fast.
- Says **how**.

**Analogy:** the function is the answer key to every possible question. The program is a student who has to work the answers out with a pencil. Two different students (programs) can produce the same answers.

QUICK CHECK 2 · VOTE WITH FINGERS

# Which statement is true?

**A.** The agent function and the agent program are the same thing.

**B.** Two different programs can implement the same agent function.

**C.** The agent function is written in Python.

**D.** $P^*$ means exactly one percept.

**B.** The function describes behavior. Many programs can produce that same behavior.

C is wrong: the function is a mathematical mapping, not code. D is wrong: $P^*$ means histories of any length.

RATIONALITY · THINK FIRST

# Was this decision foolish?

You check the forecast: only 5% chance of rain. You leave your umbrella at home. It pours, and you get soaked.

**YOUR TURN (60 seconds, ask your neighbour)**
Were you irrational? Write **yes** or **no** and one reason.

**No.** You made a sensible choice with the information you had. Bad luck is not bad reasoning.

**NEW WORD · Rational:** choosing the action that is expected to do best, given what you know. It does not mean you are never surprised.

RATIONAL MEANS FOUR THINGS TOGETHER

# What decides a good action?

FACTOR 1

## The performance measure

What counts as success.

Vacuum: points for clean cells

FACTOR 2

## Prior knowledge

What the agent knew before it started.

Vacuum: the room is a grid

FACTOR 3

## The available actions

What it is able to do.

Vacuum: suck, move, do nothing

FACTOR 4

## The percept sequence

Everything it has sensed so far.

Vacuum: cells seen so far

**Rational action** = the action expected to maximize the performance measure, given factors 2, 3 and 4.

DO NOT CONFUSE THESE

# Rational is not all-knowing

| | Rational agent | Omniscient agent |
|---|---|---|
| Maximizes | expected performance | actual performance |
| Uses | what it has sensed and knows | the true outcome of every action |
| Possible? | Yes, we can build it | No, nobody can see the future |
| Umbrella story | Left it at home: 5% rain | Would have known it would pour |

**Take-away:** we judge an agent's **reasoning**, not its luck. A rational agent can still lose, as long as it chose well with what it knew.

QUICK CHECK 3 · VOTE WITH FINGERS

# Was the trading agent irrational?

A trading agent buys a share after careful analysis of prices. Minutes later an unpredictable event crashes the price, and the agent loses money.

**A.** Yes, because it lost money.

**B.** No, it chose the best action given what it knew.

**C.** Yes, it should have known about the event.

**D.** We cannot say anything about it.

**B.** Nobody could have known. Rationality judges the decision, not the outcome. C asks for omniscience.

THE MEASURE SHAPES THE BEHAVIOR

# Be careful what you reward

Suppose we reward the vacuum for **each time it sucks up dirt**. A clever agent finds a loophole.

**YOUR TURN (45 seconds)**
What could the agent do to collect the most points without making the room cleaner?

**Suck, dump the dirt back on the floor, suck again.** The score climbs but the floor never gets clean.

**Fix:** reward the state of the world (clean floor), not the agent's activity. Our lab does exactly this:

suck a dirty cell +10 · suck a clean cell −1 · any move −1 (even into a wall) · noop 0

QUICK CHECK 4 · TRUE OR FALSE

# Three statements, one finger each

**1.** A rational agent never makes a mistake.

**2.** What the agent has sensed so far can change which action is rational.

**3.** If we change the performance measure, a different action may become the rational one.

**1. False.** It can be unlucky or wrongly informed. **2. True.** It is one of the four factors. **3. True.** Success is defined by the measure.

NEW WORD · PEAS

# Describe the problem before the robot

An engineer writes a shopping list before cooking. For an agent, the list has four parts:

**P**

**Performance measure**

How we score success.

**E**

**Environment**

The world it works in.

**A**

**Actuators**

How it acts.

**S**

**Sensors**

How it senses.

**Rule of thumb:** if you cannot fill all four boxes, you do not yet understand the problem well enough to build an agent for it.

WORKED EXAMPLE

# Autonomous delivery agent

| PEAS part | Description |
|---|---|
| P Performance | Safety, on-time delivery percentage, energy efficiency, customer satisfaction |
| E Environment | Urban streets, sidewalks, weather conditions, moving pedestrians |
| A Actuators | Electric drive motors, steering servos, indicator lights, notification speaker |
| S Sensors | Cameras, LiDAR, ultrasonic proximity sensors, GPS module, motion sensor (IMU) |

Watch the pattern: **P** is a list of things we want, **E** is the world, **A** is what it can change, **S** is what it can read.

YOUR TURN · FILL THE PEAS

# A smart greenhouse watering agent

A greenhouse has plants, soil sensors and a water tap. An agent should keep the plants healthy while saving water.

| PEAS part | Your ideas (2 minutes in pairs) |
|---|---|
| P Performance | |
| E Environment | |
| A Actuators | |
| S Sensors | |

**P:** healthy plants, water used, energy used. **E:** plants, soil, air, weather, the greenhouse. **A:** water valve, vent motors, grow lights. **S:** soil moisture probe, temperature and humidity sensors, light sensor.

Your answers may differ and still be right. Check that nothing sits in the wrong box.

FROM THE LAB GUIDE

# Four more PEAS descriptions

| Agent | Performance | Environment | Actuators | Sensors |
|---|---|---|---|---|
| Vacuum cleaner | Cleanliness, energy, time | Grid cells, dirt, walls | Motors, suction, steering | Bump, dirt and location sensors |
| Automated taxi | Safe arrival, speed, legal, comfort | Roads, traffic, pedestrians, weather | Steering, accelerator, brake, horn | Cameras, LiDAR, radar, GPS, speedometer |
| Medical assistant | Diagnostic accuracy, treatment efficiency, patient outcome | Patient records, lab tests, symptom reports | Diagnostic reports, treatment plans | Keyboard input, health-record feeds |
| Trading agent | Return, risk-adjusted return, low slippage | Exchange feeds, news, order books | Limit, market and cancel orders | Live price feeds, order-depth API |

Same four boxes, very different worlds. The vacuum row is the world we build in the lab.

QUICK CHECK 5 · WHICH LETTER?

# Sort each item into P, E, A or S

1. “On-time delivery percentage” for a delivery agent — P E A S
2. “Pedestrians crossing the road” for a taxi agent — P E A S
3. “LiDAR” on a delivery robot — P E A S
4. “Brake” on a taxi agent — P E A S

**1** → **P** (a success measure) · **2** → **E** (part of the world) · **3** → **S** (reads the world) · **4** → **A** (changes the world).

FOUR KINDS OF AGENT

# From simple to smart

Agents differ in how deep their thinking goes. Each level adds one new ability.

LEVEL 1

### Simple reflex

Reacts to what it sees right now. No memory.

LEVEL 2

### Model-based

Keeps an internal picture of the world, even the parts it cannot see.

LEVEL 3

### Goal-based

Knows where it wants to end up and picks actions that lead there.

LEVEL 4

### Utility-based

Scores outcomes with numbers and trades one goal against another.

Today we build levels 1 and 2 in the lab. Level 3 arrives next week with path-finding.

LEVEL 1 · SIMPLE REFLEX AGENT

# If this, then that

It uses **condition-action rules** on the current percept only. The vacuum in the lab uses two rules:

```
if status == "dirty":
    return "suck"
else: # pick a random direction
    return random.choice([
        "move_north", "move_south", "move_east", "move_west"])
```

**Weakness:** it has no memory. After cleaning a cell it wanders at random, happily walking back over cells that are already clean, and it never learns that the room is finished.

LEVEL 2 · MODEL-BASED AGENT

# It remembers what it cannot see now

THE AGENT'S INTERNAL MAP (3×3 room)

| | | |
|---|---|---|
| clean | clean | ? |
| clean | dirty | ? |
| ? | ? | ? |

**NEW WORD · State:** what the agent believes about the world, updated after every percept.

The sensor only reports the **current cell**. The map above is built from history.

"?" means **not yet seen**, which is different from clean.

**Payoff:** the agent can head for a "?" cell instead of wandering over clean ones, and once nothing is dirty or unknown it can stop moving and save energy.

QUICK CHECK 6 · WHICH LEVEL?

# Name the architecture

**1.** A thermostat switches the heater on whenever the temperature is below 20°C.

**2.** A vacuum keeps a map of cells it already cleaned so it avoids them.

Fingers: 1 = simple reflex · 2 = model-based · 3 = goal-based · 4 = utility-based

**1 → Simple reflex.** One rule on the current reading, no memory.

**2 → Model-based.** The map is internal state built from the percept history.

LEVEL 3 · GOAL-BASED AGENT

# It knows where it wants to go

A goal-based agent keeps a model **and** a goal. It asks: “which sequence of actions gets me there?”

| Situation | Reflex answer | Goal-based answer |
|---|---|---|
| Delivery robot, address 12 Palm Road | Drive forward until something blocks you | Plan the route to number 12, then follow it |
| Vacuum, bin is full | Keep moving at random | Go to the dock, empty it, return |

Planning a route is its own topic. Next week we meet the tools: A* and Dijkstra.

LEVEL 4 · UTILITY-BASED AGENT

# When two good things compete

Reaching the goal is not enough if there are many ways to get there. A **utility** is a number saying how happy we are with an outcome.

```
utility = 100 - 3 × minutes - 5 × risk
```

| Route | Minutes | Risk (0–10) | Utility |
|---|---|---|---|
| A, fast road | 10 | 4 | 100 − 30 − 20 = 50 |
| B, safer road | 14 | 1 | 100 − 42 − 5 = 53 |

**Route B wins.** Both reach the destination, so a goal-based agent sees a tie. The utility function lets the agent trade a few minutes for safety.

THE CONTROL LOOP IN PYTHON

# Sense, decide, act, repeat

```
class SimpleReflexAgent:
    def __init__(self):
        self.rules = {
            "obstacle_ahead": "stop_and_turn",
            "clear_path": "move_forward",
            "battery_low": "return_to_dock",
        }
    def select_action(self, percept):
        status = percept.data.get("status", "clear_path")
        return self.rules.get(status, "move_forward")

def agent_loop(agent, environment):
    while environment.is_active():
        percept = environment.get_percept()
        action = agent.select_action(percept)
        environment.apply_action(action)
```

> **NEW WORD · Control loop:** the cycle an agent repeats for as long as it runs.

**1. get_percept** is the sensor reading.

**2. select_action** is the agent program.

**3. apply_action** is the actuator at work.

**.get(status, default)** means: look it up, and if it is missing use the default.

YOUR TURN · PREDICT, THEN REVEAL

# What does select_action return?

Using the class on the last slide, write the action for each percept. You have 60 seconds.

| # | Percept data | Your prediction |
|---|---|---|
| 1 | {"status": "obstacle_ahead"} | |
| 2 | {"status": "battery_low"} | |
| 3 | {"status": "rain"} | |
| 4 | {} (no status at all) | |

**1** stop_and_turn · **2** return_to_dock · **3** move_forward · **4** move_forward

Rows 3 and 4 trip people up: “rain” is not in the rules, and an empty percept falls back to “clear_path”. Both end at the default, move_forward. A default is a design decision, and here it could be unsafe.

QUICK CHECK 7 · VOTE WITH FINGERS

# What would make it model-based?

We add one line to __init__: **self.visited = set()**, and every call to select_action adds the current location to it. The agent then avoids visited cells. What kind of agent is it now?

**A.** Still simple reflex

**B.** Model-based

**C.** Utility-based

**D.** Not an agent

**B.** It now keeps internal state (visited cells) built from past percepts, and uses it to choose actions.

RECAP

# Four architectures at a glance

| Architecture | Memory of the past? | Has a goal? | Trades off with numbers? |
|---|---|---|---|
| Simple reflex | No | No | No |
| Model-based | Yes | No | No |
| Goal-based | Yes | Yes | No |
| Utility-based | Yes | Yes | Yes |

Each level builds on the one before. In the lab you climb one step: from level 1 to level 2.

BREAK

# Five minutes

Back at 1:15. Stretch, drink some water.

While you walk, think: **how could a vacuum avoid cleaning the same cell twice?** You will build the answer in the lab.

LAB 1 · BUILDING PEAS AGENT ARCHITECTURES

# Three parts, one vacuum world

You describe a problem, build a smarter agent, then measure whether it really is smarter.

PART 1 · about 8 min

## PEAS analysis

Write a full PEAS description for an AI automated code reviewer.

PART 2 · about 20 min

## ModelBasedAgent

Extend the starter SimpleReflexiveAgent so it remembers clean and dirty cells.

PART 3 · about 7 min

## Benchmark

Run 100 trials and compare the average score of both agents.

**Graded lab.** Your code must pass every test in test_lab1.py before you open your pull request to the course grading repository. Work in pairs, one types and one reads, and swap halfway.

PART 1 · YOUR TURN

# PEAS for an AI code reviewer

A code reviewer agent reads a developer's proposed change to a program and leaves feedback before the code is merged.

| PEAS part | Prompt to think about | Your answer (8 minutes) |
|---|---|---|
| P | How would we know a review was good? | |
| E | What does the reviewer work inside? | |
| A | What can it do to the change? | |
| S | What can it read? | |

Remember the rule: if it reads, it is a sensor; if it changes something, it is an actuator.

PART 1 · SAMPLE ANSWER

# One good PEAS for a code reviewer

| PEAS part | Sample description |
|---|---|
| P Performance | Real bugs found, few false alarms, review speed, developer acceptance of suggestions |
| E Environment | Pull requests, the code repository, the team’s style guide, automated test results, developers |
| A Actuators | Inline comments, approve or request changes, suggested patches, a pass or fail status check |
| S Sensors | The code diff, file contents, test and lint output, repository history, project documentation |

Yours may differ. Check: is every item in the right box, and could you measure each item in P?

BEFORE CODING · READ THE STARTER CODE

# Three questions about the world

**1.** What exactly does the agent receive each step?

**2.** What reward does moving into a wall earn?

**3.** Can the percept tell the agent about other cells in the room?

```
reward: suck dirty +10 · suck clean -1
move (any direction) -1 · noop 0 · 3×3 grid, 30% dirty
```

**1.** A dictionary with the agent's **location** and the **status** of that one cell. **2.** −1: the position stays inside the grid, but the move still costs energy. **3.** No. Only the current cell. That is exactly why memory helps.

PART 2 · DESIGN ON PAPER FIRST

# Plan before you code

| Design question | Your plan (3 minutes) |
|---|---|
| 1. What does the agent remember? | |
| 2. When does the memory change? | |
| 3. How does it pick a move? | |
| 4. What if no cell is left to visit? | |

**1.** A dictionary from (x, y) to “clean” or “dirty”; unseen cells are simply missing. **2.** On every call, from the new percept; and after a suck the cell is now clean. **3.** Step toward a cell not yet in the dictionary. **4.** Return “noop”, which costs nothing.

Also decide: does the agent know the grid size? That is **prior knowledge**, factor 2 of rationality.

PART 2 · SKELETON ONLY

# Your ModelBasedAgent

```
class ModelBasedAgent(SimpleReflexiveAgent):
    def __init__(self, name="ModelBasedAgent"):
        super().__init__(name)
        self.world = {} # (x, y) → clean or dirty

    def perceive_and_act(self, percept):
        # TODO 1: record what this cell looks like
        # TODO 2: if it is dirty → suck, and remember
        # that it will be clean afterwards
        # TODO 3: otherwise move toward a cell you
        # have not seen; if none → noop
        ...
```

### SELF-TESTS BEFORE THE OFFICIAL TESTS

- On a room that is all clean, it never returns “suck”.
- It never sucks twice in the same cell.
- It always returns one of the six valid action strings.
- On the same room it scores higher than the reflex agent, on average.

> Run test_lab1.py often. The official tests are the final judge.

PART 3 · BENCHMARK

# Is the smarter agent really better?

```
def run_trials(agent_class, trials=100, steps=20):
    totals = []
    for t in range(trials):
        # TODO: same random floor for both agents
        env = Environment(3, 3)
        agent = agent_class()
        for _ in range(steps):
            # percept → action → reward → update score
        totals.append(agent.total_score)
    return # TODO: the average
```

### YOUR TURN · PREDICT FIRST

With 20 steps, will the reflex agent's average be around −20, 0 or +20? And the model-based agent's?

What happens to the reflex score if you run 50 steps instead? Why?

**Typical results** (our reference agent, 3×3 room, 100 trials; yours will differ by a few points): **20 steps:** reflex ≈ 0 to 2, model-based ≈ 18 to 21. **50 steps:** reflex ≈ −22 to −25, model-based ≈ 18 to 20.

Why? Once the room is clean the reflex agent keeps moving at −1 per step. The model-based agent stops with noop. Same score from cleaning, very different cost of finishing.

STUCK? TOTALLY NORMAL

# The five most common lab bugs

| What you see | Likely cause | Fix |
|---|---|---|
| The agent sucks again and again on a clean cell | Forgot to record the cell as clean after a suck | Update the memory when you choose “suck” |
| It never moves, or stops too early | Treated unseen cells as clean | Unseen means missing from the dictionary, not clean |
| AttributeError: location | Skipped super().__init__() | Call the parent constructor first |
| Test fails: invalid action | Returned None when no move was found | Return “noop” explicitly |
| Benchmark gives the same score for both agents | Same seed used wrongly, or the wrong class created | Seed before the room, create agent_class() each trial |

Still stuck after two minutes? Raise a hand, or write the exact error on the board.

EXIT QUIZ · 5 MINUTES, ON YOUR OWN

# Six questions, one letter each

**1.** An agent perceives through ___ and acts through ___.

A actuators; sensors · B sensors; actuators · C rules; memory · D PEAS; code

**2.** The agent function f: P* → A maps…

A one percept to one action · B a whole percept history to an action · C actions to percepts · D PEAS to code

**3.** A rational agent is one that…

A never errs · B maximizes expected performance given what it knows · C knows every outcome · D acts fastest

**4.** “On-time delivery percentage” belongs to which PEAS letter?

A P · B E · C A · D S

**5.** A vacuum that keeps a map of cells already cleaned is…

A simple reflex · B model-based · C goal-based · D utility-based

**6.** An agent that scores outcomes with numbers to trade off competing goals is…

A simple reflex · B model-based · C goal-based · D utility-based

ANSWER KEY

# Check your own paper

**1. B.** Sensors take information in; actuators act on the world.

**2. B.** The function maps an entire history of percepts to an action.

**3. B.** Rational means best expected performance, not knowing the future.

**4. A.** A success measure is Performance.

**5. B.** Internal state built from past percepts makes it model-based.

**6. D.** A utility function turns outcomes into numbers so goals can be traded off.

REFLECTION · TAKE THESE HOME

# Can you answer these in your own words?

**1.** Why is a rational agent allowed to fail?

**2.** What is the difference between the agent function and the agent program?

**3.** Why did the model-based agent beat the reflex agent in the benchmark, and what did it cost to achieve that?

**MUDDIEST POINT:** on a slip of paper, write the one idea from today that is still unclear. We start next time with the most common one. **Homework:** finish Parts 2 and 3 and make test_lab1.py pass before you submit.

INTELLIGENT AGENTS

# Thank you. Questions?

Today a robot vacuum became a formal idea: sense, decide, act, with a measure of success and a memory that can beat luck.

Dr.-Ing. Chukwuka Victor Obionwu